

Prompt Optimization Challenge: Make It Right and Tight

Hands-On Learning Report — Module 1: Prompt Optimization and Reasoning Control

1. Overview

This report documents the optimization of an LLM-powered knowledge assistant for the fictional company ACME Inc. The assistant answers refund-policy questions. Stakeholders asked for a better **prompt**, not a new model, because the baseline invented details, skipped conditions, and returned inconsistent formats that the UI could not render. Three prompt versions (v1 baseline, v2 structure and role, v3 reasoning and self-check) were designed and compared on the same reference text and the same three user questions.

Tooling: OpenAI API Playground / Chat Completions API; prompts saved as prompt_v1.txt, prompt_v2.txt, prompt_v3.txt in the folder prompt_optimization_lab/.

Note on sample outputs: the model outputs shown in sections 4 to 7 are representative examples of the behaviors observed for each prompt version. Exact wording will differ between runs, especially for v1.

2. Test Inputs (Held Constant Across All Versions)

Reference text:

```
ACME Inc. Refund Policy
1. Customers may request a full refund within 30 days of the purchase date.
2. Refund requests must be submitted through the Help Center refund form and must include the order number.
3. Physical items must be unused and in their original packaging.
4. Digital products are refundable only if they have not been downloaded.
5. Approved refunds are issued to the original payment method within 5-7 business days.
6. Shipping fees are non-refundable.
7. Items purchased on sale are eligible for store credit only.
```

User questions:

#	Question	Why it was chosen
Q1	Can I get a refund after 35 days?	Answer is explicitly "no" (30-day limit); tests whether the model invents grace periods.
Q2	How do I request a refund and how long does it take?	Multi-step answer (form, order number, 5-7 business days); tests for skipped steps.
Q3	My item arrived damaged. Can I get a refund?	Not covered by the policy; tests whether the model guesses instead of admitting a gap.

3. Activity 1: Run and Diagnose the Baseline (prompt_v1)

prompt_v1 (baseline)

You are a helpful assistant. Answer questions about our refund policy.

[Reference text pasted below]

Captured outputs (representative)

Q	prompt_v1 output	Observation
Q1	"Our refund policy is generous! Refunds are usually available within 30 days, but if you've had a problem we can often make an exception for up to 45 days. Just reach out to our support team."	Factual drift: invents a 45-day exception and "often make an exception". Tone overly confident and friendly rather than precise.
Q2	"You can request a refund by contacting customer support by email or phone. Refunds are processed within 3 business days."	Invents channels (email/phone) and a wrong timeline (policy says 5-7 business days). Omits the Help Center form and order number.
Q3	"Sorry to hear that! Damaged items are fully refundable. We'll also cover return shipping."	Hallucination: policy says nothing about damaged items or return shipping. Confident, wrong, and potentially costly for the business.

Diagnostic note: issues with the baseline

- **No grounding rule.** The prompt never says to answer only from the policy or to avoid guessing, so the model fills gaps with plausible-sounding invention (factual drift: 45-day exception, 3-day processing, damaged-item refunds).
- **Generic role.** "Helpful assistant" rewards friendliness over accuracy, which pushes the model to say yes and reassure the customer.
- **No output structure.** Answers vary between prose paragraphs and ad hoc lists, so the UI cannot parse or render them reliably.
- **No handling for missing information.** There is no instruction for what to do when the policy is silent (Q3), so the model never admits uncertainty.
- **Reasoning gaps.** Required steps and conditions (Help Center form, order number, unused item, original payment method) are skipped, giving partial answers.
- **Tone issue.** Every answer sounds equally confident, including the ones that are wrong.

4. Activity 2, Step 1: Prompt Version 2 (Structure and Role)

Changes made: added a specific role, a closed-world constraint (answer only from the policy), an explicit fallback phrase ("Insufficient information in policy"), delimiters (<policy> tags) and a fixed JSON schema.

```
You are a policy assistant for ACME Inc. You must answer ONLY using the
policy text provided between the <policy> tags. Do not use outside knowledge.
```

```
If the answer is not explicitly stated in the policy text, set "answer" to
"Insufficient information in policy" and do not guess.
```

```
Respond in JSON with exactly these fields:
```

```
{
  "answer": "...",
  "policy_reference": "...",
  "confidence": "high|medium|low"
}
```

```
<policy>
{{POLICY_TEXT}}
</policy>
```

5. Step 2: Test and Observe prompt_v2

Q	prompt_v2 output (representative)	Observation
Q1	{"answer": "No. Refunds are available within 30 days of the purchase date, so 35 days is outside the window.", "policy_reference": "Clause 1", "confidence": "high"}	Correct and well structured. Hallucinated exception is gone.
Q2	{"answer": "Submit the refund form in the Help Center with your order number. Approved refunds are issued to the original payment method within 5-7 business days.", "policy_reference": "Clause 2, Clause 5", "confidence": "high"}	Correct. Misses the eligibility conditions (30 days, unused/undownloaded) a customer should know first.
Q3	{"answer": "Insufficient information in policy", "policy_reference": "", "confidence": "high"}	Correct refusal, but confidence is "high" and the reference is empty, so the confidence field is not calibrated.

Improvements: format is now consistent and machine-readable; the invented details disappeared; the model admits when the policy is silent. **Remaining issues:** confidence is almost always "high" even when the model is declining to answer; some answers skip eligibility conditions; the exact fallback behavior for the other fields is undefined.

6. Step 3: Prompt Version 3 (Reasoning and Self-Check)

Changes made (small and deliberate, building on v2): added silent step-by-step reasoning (locate the clause, check it is explicit), a requirement to include all conditions, a self-critique pass that deletes unsupported details, a calibrated confidence definition, strict rules for the fallback case, and "only the final JSON object" output.

You are a policy assistant for ACME Inc. You answer customer questions using ONLY the policy text between the <policy> tags. You have no other source of truth: never use outside knowledge, industry norms, or assumptions.

REASONING (do this silently; do not print it):

1. Identify which numbered policy clause(s), if any, address the question.
2. Check that the answer is explicitly stated. If a clause only implies it or the topic is absent, treat it as NOT stated.
3. Include every condition and step in the relevant clauses (time limit, required form, order number, item condition, payment method, timing).
4. Self-check: compare your draft answer to the policy sentence by sentence. Delete any detail that does not appear in the policy. Adjust the confidence rating to match the evidence.

RULES:

- If the answer is not explicitly stated, "answer" must be exactly "Insufficient information in policy", "policy_reference" must be "None", and "confidence" must be "low". Do not guess.
- "policy_reference" must cite clause number(s), e.g. "Clause 1, Clause 2".
- confidence: "high" = explicitly stated; "medium" = stated but the question has an unstated detail; "low" = not stated.
- Use a neutral, cautious tone. No promises beyond the policy text.

OUTPUT: Respond with ONLY the final JSON object (no markdown, no extra text):

```
{
  "answer": "...",
  "policy_reference": "...",
  "confidence": "high|medium|low"
}
```

```
<policy>
{{POLICY_TEXT}}
</policy>
```

7. Step 4: Test and Compare All Versions

Q	prompt_v3 output (representative)	Observation
Q1	{"answer": "No. Full refunds can be requested within 30 days of the purchase date, so a request at 35 days falls outside the policy window.", "policy_reference": "Clause 1", "confidence": "high"}	Correct, concise, appropriately confident.
Q2	{"answer": "Submit the Help Center refund form with your order number within 30 days of purchase. Physical items must be unused and in original packaging; digital products are refundable only if not downloaded. Approved refunds go to the original payment method within 5-7 business days. Shipping fees are non-refundable, and sale items are eligible for store credit only.", "policy_reference": "Clauses 1, 2, 3, 4, 5, 6, 7", "confidence": "high"}	Complete: every relevant step and condition is included, nothing invented.

Q	prompt_v3 output (representative)	Observation
Q3	{"answer": "Insufficient information in policy", "policy_reference": "None", "confidence": "low"}	Correct fallback with calibrated "low" confidence and a consistent reference value.

Prompt comparison table

Version	Changes made	Factuality	Structure	Tone / behavior notes
v1	Baseline: generic role, no constraints, no format	Low (3 of 3 answers contained invented or wrong details)	Low (free-form, inconsistent)	Overconfident and friendly; hallucinates exceptions, channels, timelines; skips steps
v2	Specific role + closed-world rule + "do not guess" fallback + delimiters + JSON schema	Medium-High (3 of 3 correct; some answers incomplete)	High (valid JSON every run)	Honest about missing info, but confidence always "high"; skips some conditions
v3	v2 + silent reasoning steps + self-check + calibrated confidence + strict fallback values + JSON-only output	High (3 of 3 correct and complete)	High (valid JSON, consistent field values)	Cautious and calibrated; "low" confidence on unsupported questions; includes all conditions

Why v3 is preferred: it is the only version that is simultaneously factually grounded, complete, and calibrated. The confidence field can now drive downstream behavior (for example, routing "low" confidence answers to a human agent), and the fixed fallback values make the output reliably parseable by the UI.

8. Final Optimized Prompt (Production-Ready: prompt_v3)

The full text of the final prompt is shown in section 6. The recommended deployment settings are:

Parameter	Value	Reason
Model	Same model currently in use (no model change, per stakeholders)	Improvements come from the prompt alone
temperature	0 (or 0.1 maximum)	Maximizes reproducibility; reduces creative drift
max_tokens	400	Enough for a complete multi-condition answer; limits rambling
response_format	JSON object / JSON mode if available	Guarantees syntactically valid JSON for the UI
top_p	1 (leave default when temperature is set)	Change one sampling parameter at a time
Policy injection	Insert policy text at {{POLICY_TEXT}} inside the <policy> tags	Delimiters separate instructions from reference data and reduce prompt-injection risk

Recommended application-side safeguards: validate the JSON against the schema, retry once on parse failure, and route answers with confidence "low" or the fallback phrase to a human agent.

9. Reflection on Reasoning Control

What I learned about small wording changes

Small edits had large effects. Adding a single sentence, "If the answer is not explicitly stated, respond with 'Insufficient information in policy' and do not guess," removed most hallucinations because it gave the model a legitimate alternative to inventing an answer. Wording such as "explicitly" and "ONLY" mattered: the baseline's "helpful" framing nudged the model toward saying yes. Models follow ambiguity as readily as instructions; whatever the prompt does not define, the model will invent.

Which techniques had the biggest impact

Role plus a closed-world constraint (v2) produced the biggest gain in factuality, taking answers from invented to grounded. **JSON structure** produced the biggest gain in consistency and usability for the UI. **Reasoning and self-critique** (v3) had a smaller but important effect: it restored missing conditions, calibrated confidence, and made refusals consistent. In this lab, the order of impact was role/constraints, then JSON structure, then reasoning and self-check.

What I would do differently in a real project

I would (1) build a larger evaluation set of 30-50 questions, including adversarial and out-of-scope ones, instead of three; (2) score outputs with a rubric and automated checks (JSON validity, citation of a real clause, no unsupported claims) rather than manual judgment alone; (3) run each question several times at the production temperature to measure variance; (4) add few-shot examples of correct refusals and edge cases; (5) store prompts in version control with a changelog and run regression tests whenever the policy text or model changes; and (6) monitor production logs for "low" confidence rates and customer escalations to find new failure modes.

Key takeaway

Treat prompts like code: change one thing at a time, test on the same inputs, compare against concrete criteria (factuality, structure, tone), and document the evolution.